

STM32F103C8 — INTERRUPT & PWM REGISTER + PROGRAM SYNTAX NOTES

Register-level programming notes from the PB12 EXTI + TIM2 PWM brightness project

Target: STM32F103C8T6 Blue Pill | **Approach:** Register-level C | **HAL:** Not used

1. Final Project Architecture

The project uses PB12 as a button input. A falling edge on PB12 triggers EXTI12. The shared EXTI15_10 interrupt handler then changes TIM2 Channel 1 compare value (CCR1). TIM2 generates the PWM waveform in hardware and outputs it on PA0 (TIM2_CH1).

```
PB12 button
  ↓
EXTI12
  ↓
NVIC: EXTI15_10_IRQn
  ↓
EXTI15_10_IRQHandler()
  ↓
change TIM2->CCR1
  ↓
TIM2 PWM (CH1)
  ↓
PA0
  ↓
LED brightness
```

2. RCC — Clock Enable Registers

RCC->APB2ENR enables GPIOA, GPIOB and AFIO. **RCC->APB1ENR** enables TIM2.

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; // GPIOA clock
RCC->APB2ENR |= RCC_APB2ENR_IOPBEN; // GPIOB clock
RCC->APB2ENR |= RCC_APB2ENR_AFIOEN; // AFIO clock
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN; // TIM2 clock
```

3. GPIO Registers

PA0 is the TIM2_CH1 output, so it must be configured as an alternate-function push-pull output. PB12 is the button input with an internal pull-up.

PA0 — GPIOA->CRL

```
GPIOA->CRL &= ~(0xFU << 0);
GPIOA->CRL |= (0xBU << 0);
```

For PA0, CRL bits [3:0] are used. 0xB = MODE 11 (50 MHz) + CNF 10 (alternate-function push-pull). 0xA could also be used for 2 MHz alternate-function push-pull.

PB12 — GPIOB->CRH

```
GPIOB->CRH &= ~(0xFU << 16);
GPIOB->CRH |= (0x8U << 16);
GPIOB->ODR |= (1U << 12);
```

PB12 is in CRH because it is pin 12. 0x8 configures input with pull-up/pull-down; setting ODR12 selects the pull-up.

4. EXTI Registers — PB12 → EXTI12

PB12 must first be mapped to EXTI12 through AFIO. Unlike IMR/FTSR/PR, EXTICR uses a 4-bit field for each EXTI line.

```
// EXTI12 uses bits [3:0] of AFIO->EXTICR[3]
// GPIOB encoding = 0001
```

```
AFIO->EXTICR[3] &= ~(0xFU << 0);
AFIO->EXTICR[3] |= (0x1U << 0);
```

Important: Do NOT use `AFIO->EXTICR[3] = (1U << 12);`. EXTI12 is represented by a 4-bit field at bits [3:0] in this register.

Enable EXTI12 and select falling edge

```
EXTI->IMR |= (1U << 12); // unmask EXTI12
```

```
EXTI->FTSR |= (1U << 12); // falling-edge trigger
EXTI->RTSR &= ~(1U << 12); // no rising-edge trigger
```

5. NVIC + ISR Syntax

EXTI10 through EXTI15 share one interrupt vector on STM32F1. Therefore PB12/EXTI12 uses EXTI15_10_IRQn and EXTI15_10_IRQHandler().

```
NVIC_EnableIRQ(EXTI15_10_IRQn);

void EXTI15_10_IRQHandler(void)
{
    if (EXTI->PR & (1U << 12))
    {
        // PB12 event

        EXTI->PR = (1U << 12); // clear pending flag
    }
}
```

Rules: Never manually call the ISR from main(). Keep the ISR short. PR is cleared by writing 1 to the corresponding bit.

6. TIM2 PWM Registers

Register	Field / bit	Purpose
TIM2->PSC	Prescaler	Divides timer clock by PSC + 1
TIM2->ARR	Auto-reload	Sets PWM period
TIM2->CCR1	Compare 1	Sets duty cycle
TIM2->CCMR1	OC1M [6:4]	110 = PWM Mode 1
TIM2->CCMR1	OC1PE bit 3	1 = CCR preload enabled
TIM2->CCER	CC1E bit 0	1 = enable CH1 output
TIM2->CCER	CC1P bit 1	1 = inverted output polarity
TIM2->EGR	UG bit 0	Generate update event
TIM2->CR1	ARPE bit 7	ARR preload enable
TIM2->CR1	CEN bit 0	Start timer

7. TIM2 Clock and PWM Calculation

Clock configuration used in this project: HSI = 8 MHz, PLL output = 8 MHz, SYSCLK = 8 MHz, APB1 = 8 MHz, and TIM2 clock = 8 MHz.

```
Timer counter frequency
= 8,000,000 / (PSC + 1)
= 8,000,000 / 8
= 1,000,000 Hz = 1 MHz
```

```
PWM frequency
= 1,000,000 / (ARR + 1)
= 1,000,000 / 1000
= 1,000 Hz = 1 kHz
```

Therefore **PSC = 7** and **ARR = 999** produce a 1 kHz PWM signal with the 8 MHz TIM2 clock.

8. TIM2 Register Configuration Syntax

Prescaler, period and initial compare value:

```
TIM2->PSC = 7;
TIM2->ARR = 999;
TIM2->CCR1 = 999; // initial ≈ 99.9% duty
```

Configure Channel 1 for PWM Mode 1:

```
TIM2->CCMR1 &= ~(0xFU << 0);
TIM2->CCMR1 |= (6U << 4) | (1U << 3);
```

Here 6 = binary 110, placed into OC1M bits [6:4]. Bit 3 enables OC1PE. CC1S remains 00, meaning CH1 is an output.

Enable CH1 and invert polarity for the active-low/sinking LED:

```
TIM2->CCER |= (1U << 0) | (1U << 1);  
// bit 0 = CC1E  
// bit 1 = CC1P
```

Load timer configuration and start:

```
TIM2->CR1 |= (1U << 7); // ARPE = 1  
TIM2->EGR = (1U << 0); // UG = 1  
TIM2->CR1 |= (1U << 0); // CEN = 1
```

Recommended order: enable ARPE before generating UG. UG is the update-generation event used to load preloaded timer values.

9. PWM Duty-Cycle Levels

With ARR = 999, there are 1000 timer counts. Approximate duty = $CCR1 / 1000 \times 100\%$.

Brightness	CCR1	Approx. duty
100%	999	99.9%
75%	750	75.0%
50%	500	50.0%
25%	250	25.0%
0%	0	0%

Note: CCR1 = 999 is approximately 100%. The exact ratio is $999/1000 = 99.9\%$.

10. Why the LED Polarity Is Inverted

The LED is wired as an active-low/sinking load. The STM32 pin sinks current when it is LOW. Therefore the PWM polarity is inverted so a larger CCR1 produces more LOW time and therefore more LED brightness.

LED wiring concept:

3.3 V → resistor → LED → PA0

PA0 LOW → current flows → LED ON
PA0 HIGH → current stops → LED OFF

With CC1P = 1:
CCR1 ↑ → active LOW time ↑ → brightness ↑

11. Interrupt Changes CCR1 — Core Application Logic

The interrupt should change the compare value; it should not generate the PWM waveform with delays. TIM2 continues generating PWM in hardware.

```
void EXTI15_10_IRQHandler(void)  
{  
    if (EXTI->PR & (1U << 12))  
    {  
        // Example:  
        TIM2->CCR1 = 750;  
  
        EXTI->PR = (1U << 12);  
    }  
}
```

For the full brightness sequence, maintain a small level/index and select CCR1 values such as 999 → 750 → 500 → 250 → 0 on successive button events.

12. Useful GPIO Syntax You Have Used

```
// Output LOW  
GPIOC->BRR = (1U << 15);  
  
// Output HIGH  
GPIOC->BSRR = (1U << 15);  
  
// Read PB12  
if (!(GPIOB->IDR & (1U << 12)))
```

```

{
    // PB12 is LOW
}

```

Remember: CRL/CRH configure GPIO pins. BSRR/BRR control output state. IDR is read for input state; do not use IDR with |= or &= to try to modify it.

13. Quick Register Cheat Sheet

Peripheral	Register	Meaning
RCC	APB2ENR	Enable GPIOA / GPIOB / AFIO clocks
RCC	APB1ENR	Enable TIM2 clock
GPIOA	CRL	Configure PA0
GPIOB	CRH	Configure PB12
GPIOB	ODR	Select PB12 pull-up
AFIO	EXTICR[3]	Map PB12 to EXTI12
EXTI	IMR	Unmask EXTI12
EXTI	FTSR	Falling-edge trigger
EXTI	PR	Pending flag; write 1 to clear
NVIC	EXTI15_10_IRQn	Enable EXTI10–15 vector
TIM2	PSC	Timer clock division
TIM2	ARR	PWM period
TIM2	CCR1	PWM duty
TIM2	CCMR1	PWM mode selection
TIM2	CCER	Channel enable + polarity
TIM2	EGR	Update generation
TIM2	CR1	ARPE + timer enable

14. Most Important Concepts to Remember

1. **EXTICR** uses 4-bit fields; EXTI12 is not simply bit 12 there.
2. **IMR**, **FTSR** and **PR** use one bit per EXTI line, so EXTI12 uses bit 12.
3. **EXTI15_10_IRQHandler()** is shared by EXTI10–15.
4. **PSC** controls timer counting speed; **ARR** controls period; **CCR1** controls duty.
5. **CCMR1** selects PWM mode; **CCER** enables the channel and selects polarity.
6. **EGR.UG** generates an update event; **CR1.CEN** starts the counter.
7. PWM is generated by the timer hardware; the ISR only changes control values.
8. Avoid long delays inside an ISR.

Milestone: GPIO → EXTI → NVIC → ISR → TIM2 → Hardware PWM → LED brightness control